TRANSACTION PROCESSING AND RECOVERY

BCSE302L – Database Systems

Part 1

CONTENTS

TRANSACTION CONCEPT

- A transaction is a *unit* of program execution that accesses and possibly updates various data items.
- E.g., transaction to transfer $50 from account A to account B:

1. **read**(*A*)
2. *A* := *A* − 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

- Two main issues to deal with:
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions

EXAMPLE

- Transaction to transfer $50 from account A to account B:
  1. read(A)
  2. A := A – 50
  3. write(A)
  4. read(B)
  5. B := B + 50
  6. write(B)

ACID PROPERTY

- To ensure integrity of the data, DB system maintain the following properties of the transactions: (ACID properties)
  - Atomicity
  - Consistency
  - Isolation
  - Durability

ACID PROPERTY

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
  - That is, for every pair of transactions T_i and T_j, it appears to T_i that either T_j, finished execution before T_i started, or T_j started execution after T_i finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

TRANSACTION OPERATIONS

- Transactions access data using two operations:
  - read(X), which transfers the data item X from the database to a local buffer belonging to the transaction that executed the read operation.
  - write(X), which transfers the data item X from the local buffer of the transaction that executed the write back to the database.

TRANSACTION OPERATIONS

- Transactions access data using two operations:
  - read(X), has following steps:
  - Find the address of the disk block that contains item X.
  - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
  - Copy item X from the buffer to the program variable named X.

TRANSACTION OPERATIONS

- Transactions access data using two operations:
  - write(X), has following steps:
  - Find the address of the disk block that contains item X.
  - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
  - Copy item X from the program variable named X into its correct location in the buffer.
  - Store the updated block from the buffer back to disk (either immediately or at some later point in time).

EXAMPLE ACID PROPERTY

- Atomicity requirement:
  - If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
    - Failure could be due to software or hardware
  - The system should ensure that updates of a partially executed transaction are not reflected in the database
- Durability requirement:
  - Once the user has been notified that the transaction has completed (i.e., the transfer of the $50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

EXAMPLE ACID PROPERTY

- Consistency requirement:
  - The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
  - Explicitly specified integrity constraints such as primary keys and foreign keys
  - Implicit integrity constraints
    - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
  - A transaction must see a consistent database.
  - During transaction execution the database may be temporarily inconsistent.
  - When the transaction completes successfully the database must be consistent
  - Erroneous transaction logic can lead to inconsistency

EXAMPLE ACID PROPERTY

- Isolation requirement
- if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum A + B will be less than it should be).

T1	T2
1. read(A)	
2. A := A – 50	
3. write(A)	
	read(A), read(B), print(A+B)
4. read(B)	
5. B := B + 50	
6. write(B)	

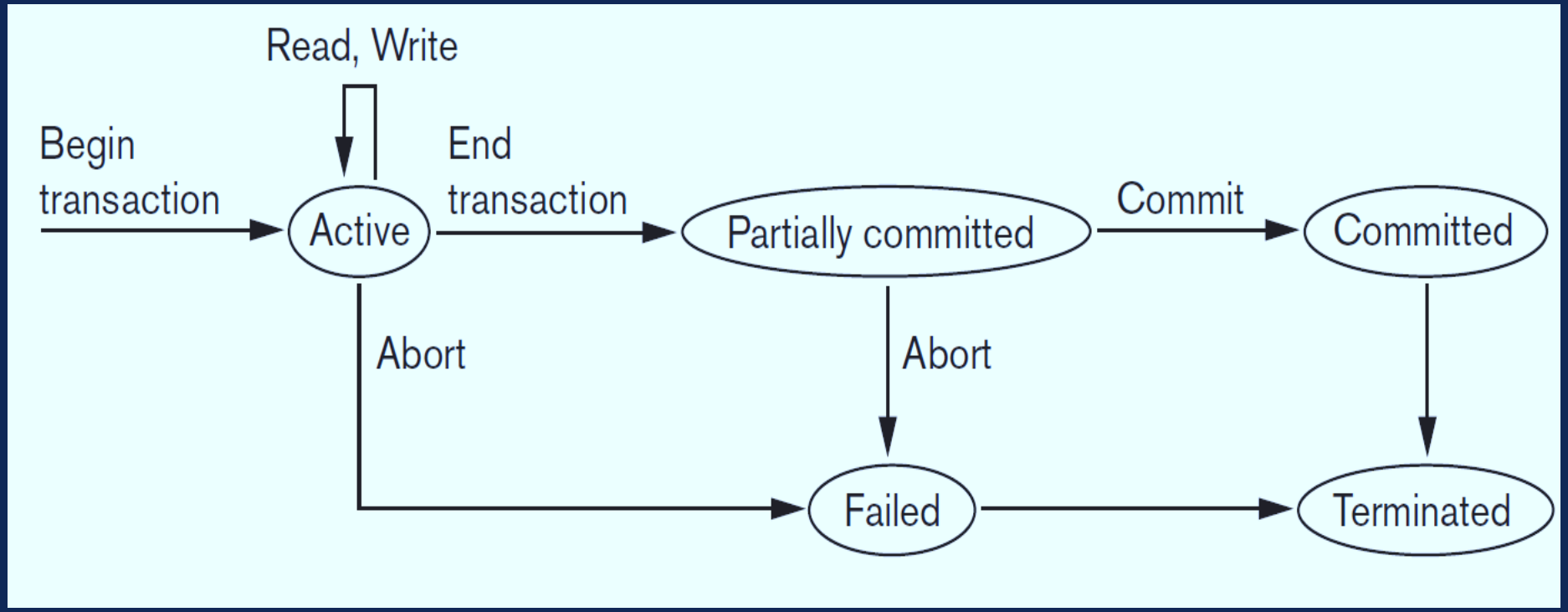
EXAMPLE ACID PROPERTY

- Isolation requirement
- Isolation can be ensured trivially by running transactions serially
  - That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.

TRANSACTION STATE

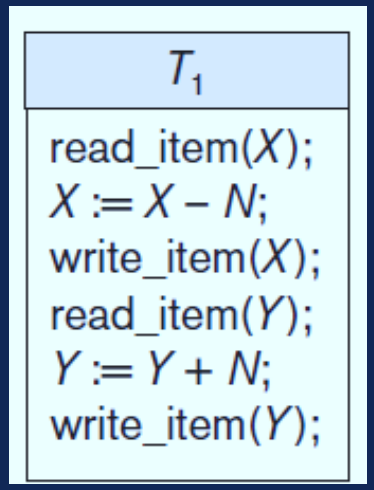
- Active – the initial state; the transaction stays in this state while it is executing
- Partially committed – after the final statement has been executed.
- Failed -- after the discovery that normal execution can no longer proceed.
- Aborted – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
  - Restart the transaction - Can be done only if no internal logical error
  - Kill the transaction
- Committed – after successful completion.

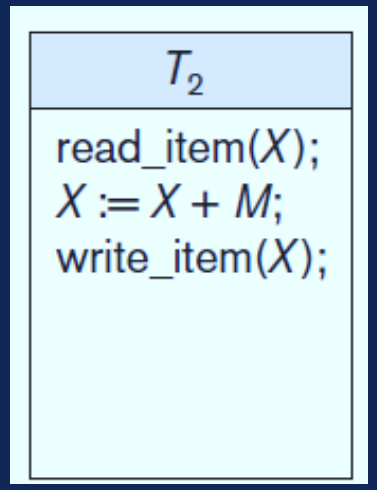
TRANSACTION STATE



CONCURRENT CONTROL IS NEEDED?

- Types of problem we may encounter with following Transactions T1 and T2

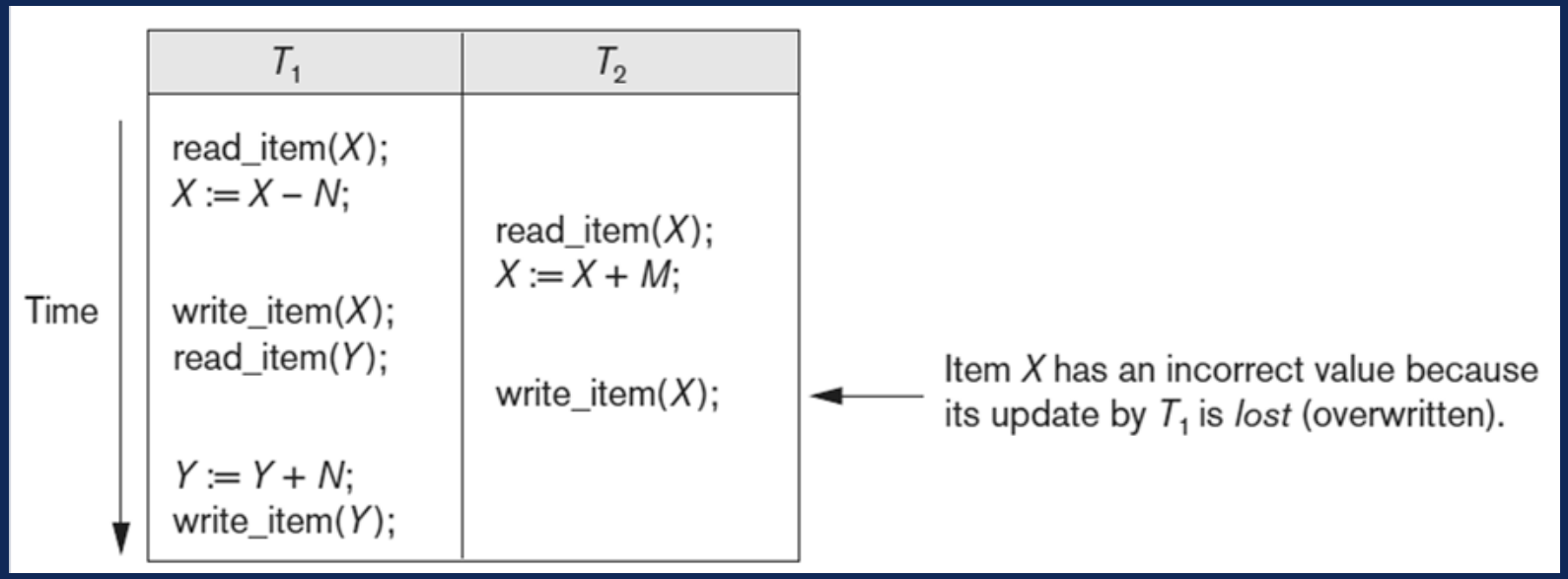




- The Lost update problem
- Temporary update (or dirty read) problem
- Incorrect summary problem
- Unrepeatable Read problem.

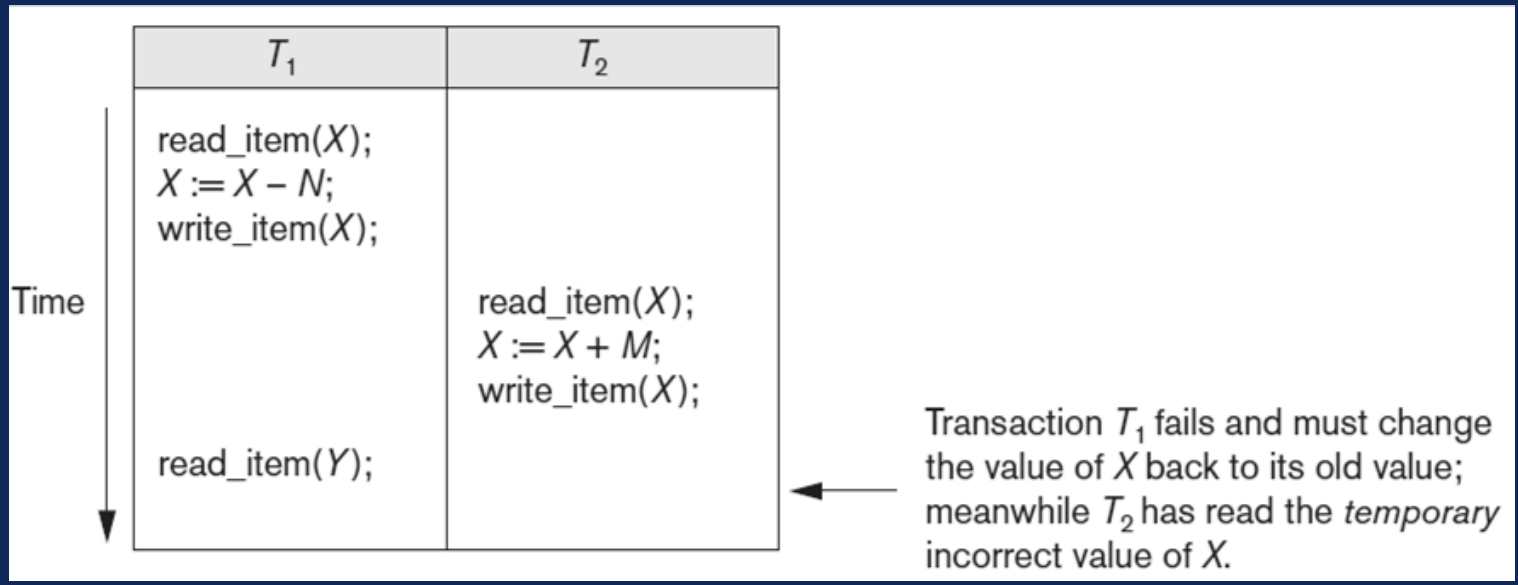
CONCURRENT CONTROL IS NEEDED?

- The Lost update problem
- This problem occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database items incorrect.



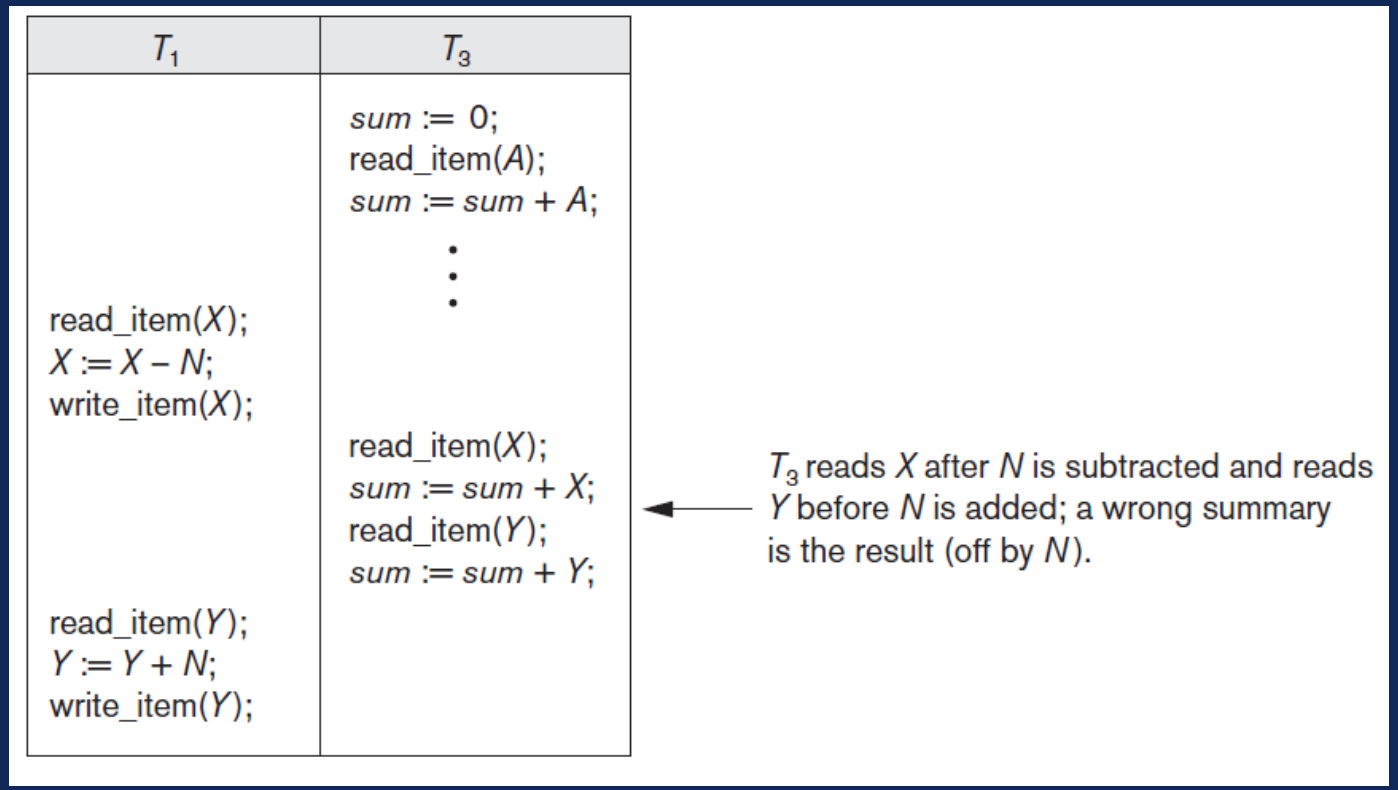
CONCURRENT CONTROL IS NEEDED?

- Temporary update (or dirty read) problem
- This problem occurs when one transaction updates a database item and then the transaction fails for some reason. Meanwhile, the updated item is accessed (read) by another transaction before it is changed back to its original value.



CONCURRENT CONTROL IS NEEDED?

- Incorrect summary problem
- If one transaction is calculating an aggregate summary function on a number of database items while other transactions are updating some of these items, the aggregate function may calculate some values before they are updated and others after they are updated.



RECOVERY IS NEEDED?

- DBMS must not permit some operations of a transaction T to be applied to the database while other operations of T are not, because the whole transaction is a logical unit of database processing.
- If a transaction fails after executing some of its operations but before executing all of them, the operations already executed must be undone and have no lasting effect.

RECOVERY IS NEEDED?

- Types of Failures: Failures are generally classified as transaction, system, and media failures. There are several possible reasons for a transaction to fail in the middle of execution:
  - Computer failure (system crash) - hardware, software, or network error occurs in the computer system during transaction execution.
  - Transaction or system error - transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error
  - Local errors or exception conditions detected by the transaction – During transaction execution, certain conditions may occur that necessitate cancellation of the transaction

RECOVERY IS NEEDED?

- Types of Failures:
  - Concurrency control enforcement - The concurrency control method may decide to abort a transaction because it violates serializability, or it may abort one or more transactions to resolve a state of deadlock among several transactions.
  - Transactions aborted because of serializability violations or deadlocks are typically restarted automatically at a later time.

RECOVERY IS NEEDED?

- Types of Failures:
  - Disk failure - Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This may happen during a read or a write operation of the transaction.
  - Physical problems and catastrophes - This refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, sabotage overwriting disks or tapes by mistake, and mounting of a wrong tape by the operator.

CONCURRENT EXECUTIONS

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
  - Increased processor and disk utilization, leading to better transaction throughput
  - E.g., one transaction can be using the CPU while another is reading from or writing to the disk
  - Reduced average response time for transactions: short transactions need not wait behind long ones.
- Concurrency control schemes – mechanisms to achieve isolation
  - That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database

SCHEDULES

- Schedule – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed
  - A schedule for a set of transactions must consist of all instructions of those transactions
  - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
  - By default transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

SCHEDULE 1

- Let T1 transfer $50 from A to B, and T2 transfer 10% of the balance from A to B.
- A serial schedule in which T1 is followed by T2 :

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit

SCHEDULE 2

- A serial schedule in which T2 is followed by T1 :

T_1	T_2
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	

SCHEDULE 3

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

- In Schedules 1, 2 and 3, the sum A + B is preserved.

SCHEDULE 4

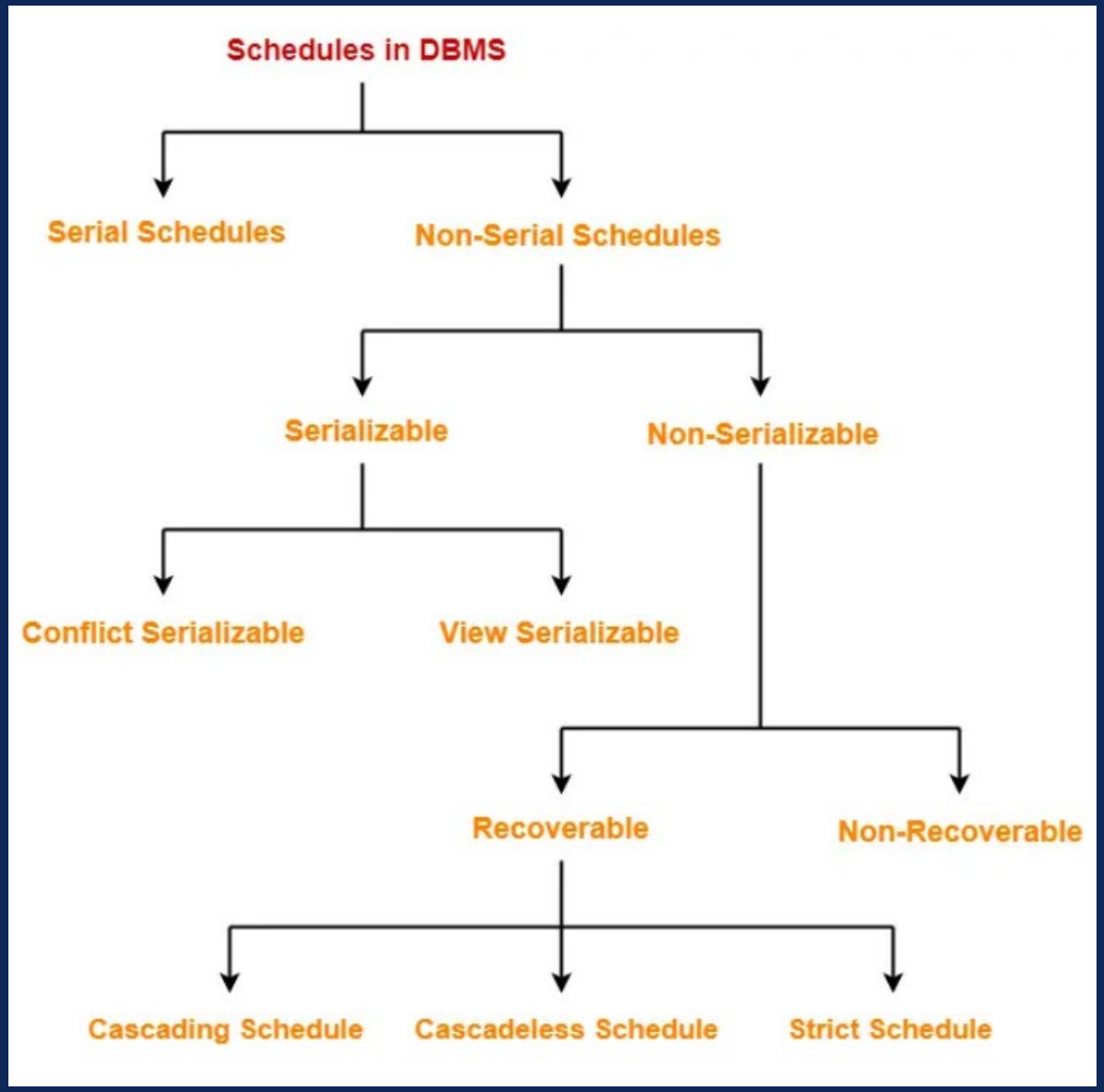
- The following concurrent schedule does not preserve the value of (*A* + *B*).

T_1	T_2
read (A)	
$A := A - 50$	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	$B := B + temp$
	write (B)
	commit

SERIALIZABILITY

- Basic Assumption – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
  - Conflict serializability
  - View serializability

TYPES OF SCHEDULE



SIMPLIFIED VIEW OF TRANSACTIONS

- We ignore operations other than read and write instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only read and write instructions.

CONFLICTING INSTRUCTIONS

- Instructions l_i and l_j of transactions T_i and T_j respectively, conflict if and only if there exists some item Q accessed by both l_i and l_j, and at least one of these instructions wrote Q.
  1. l_i = **read**(Q), l_j = **read**(Q). l_i and l_j don't conflict.
  2. l_i = **read**(Q), l_j = **write**(Q). They conflict.
  3. l_i = **write**(Q), l_j = **read**(Q). They conflict
  4. l_i = **write**(Q), l_j = **write**(Q). They conflict
- Intuitively, a conflict between l_i and l_j forces a (logical) temporal order between them.
- If l_i and l_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

CONFLICTING SERIALIZABILITY

- If a schedule *S* can be transformed into a schedule *S'* by a series of swaps of non-conflicting instructions, we say that *S* and *S'* are conflict equivalent.
- We say that a schedule *S* is conflict serializable if it is conflict equivalent to a serial schedule

CONFLICTING SERIALIZABILITY

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

Schedule 3

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 6

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

CONFLICTING SERIALIZABILITY

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule < T_3, T_4 >, or the serial schedule < T_4, T_3 >.

VIEW SERIALIZABILITY

- Let S and S' be two schedules with the same set of transactions. S and S' are view equivalent if the following three conditions are met, for each data item Q,
  1. If in schedule S, transaction T_i reads the initial value of Q, then in schedule S' also transaction T_i must read the initial value of Q.
  2. If in schedule S transaction T_i executes **read**(Q), and that value was produced by transaction T_j (if any), then in schedule S' also transaction T_i must read the value of Q that was produced by the same **write**(Q) operation of transaction T_j .
  3. The transaction (if any) that performs the final **write**(Q) operation in schedule S must also perform the final **write**(Q) operation in schedule S'.
- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.

VIEW SERIALIZABILITY

- A schedule S is view serializable if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read (Q)		
	write (Q)	
write (Q)		
		write (Q)

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has blind writes.

VIEW SERIALIZABILITY

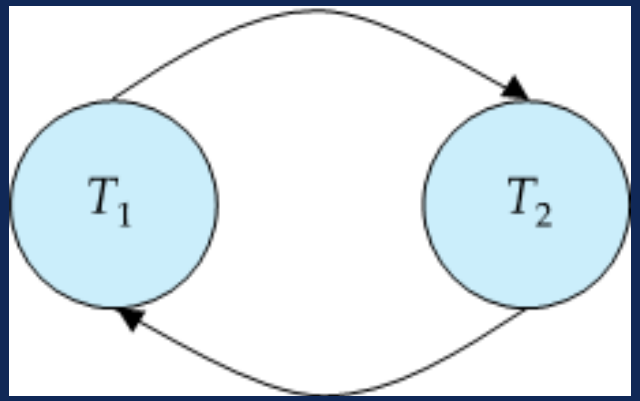
- The schedule below produces same outcome as the serial schedule < T_1, T_5 >, yet is not conflict equivalent or view equivalent to it.

T_1	T_5
read (A)	
$A := A - 50$	
write (A)	
	read (B)
	$B := B - 10$
	write (B)
read (B)	
$B := B + 50$	
write (B)	
	read (A)
	$A := A + 10$
	write (A)

- Determining such equivalence requires analysis of operations other than read and write.

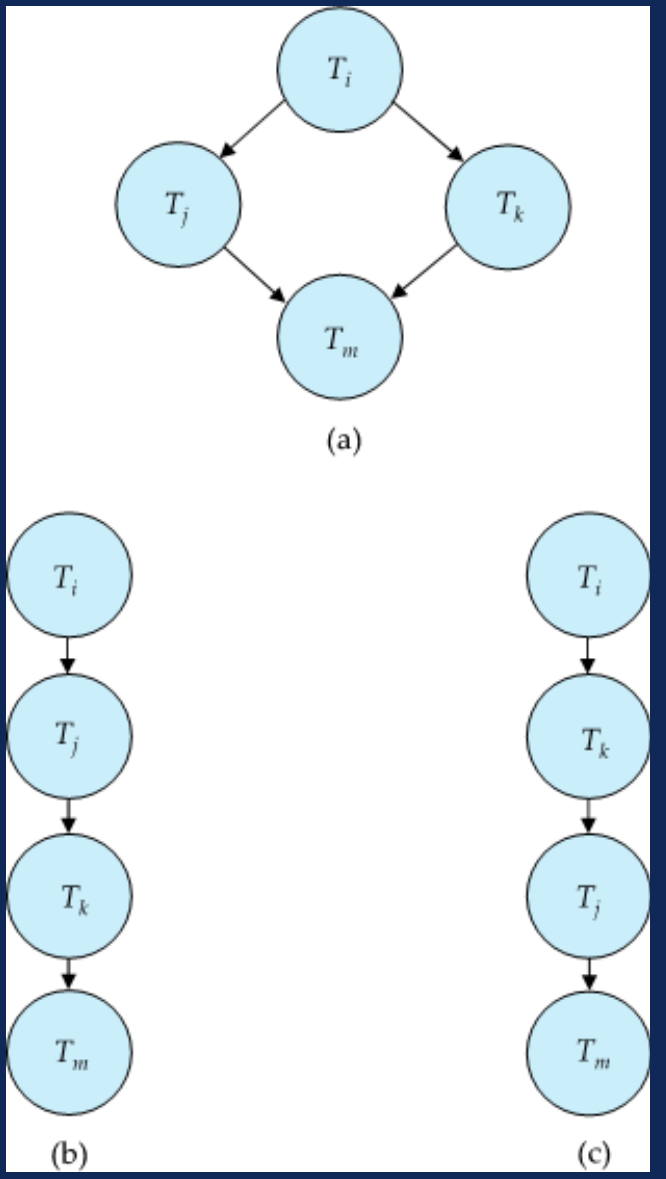
TESTING FOR SERIALIZABILITY

- Consider some schedule of a set of transactions T_1, T_2, ..., T_n
- Precedence graph — a direct graph where the vertices are the transactions (names).
- We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- Example of a precedence graph



TESTING FOR CONFLICT SERIALIZABILITY

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.
  - (Better algorithms take order $n + e$ where e is the number of edges.)
- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
- This is a linear order consistent with the partial order of the graph.



TESTING FOR VIEW SERIALIZABILITY

- The precedence graph test for conflict serializability cannot be used directly to test for view serializability.
  - Extension to test for view serializability has cost exponential in the size of the precedence graph.
- The problem of checking if a schedule is view serializable falls in the class of *NP*-complete problems.
  - Thus, existence of an efficient algorithm is *extremely* unlikely.
- However practical algorithms that just check some sufficient conditions for view serializability can still be used.

RECOVERABLE SCHEDULES

- Need to address the effect of transaction failures on concurrently running transactions.
- Recoverable schedule — if a transaction T_j reads a data item previously written by a transaction T_i, then the commit operation of T_i appears before the commit operation of T_j.
- The following schedule (Schedule 11) is not recoverable

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

CASCADING ROLLBACKS

- Cascading rollback – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A)		
read (B)		
write (A)		
	read (A)	
	write (A)	
		read (A)
abort		

  If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work

CASCADING SCHEDULES

- Cascadeless schedules — cascading rollbacks cannot occur;
  - For each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the read operation of T_j.
- Every Cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless

CONCURRENCY CONTROL

- A database must provide a mechanism that will ensure that all possible schedules are
  - either conflict or view serializable, and
  - are recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
  - Are serial schedules recoverable/cascadeless?
- Testing a schedule for serializability *after* it has executed is a little too late!
- Goal – to develop concurrency control protocols that will assure serializability.

CONCURRENCY CONTROL

- Schedules must be conflict or view serializable, and recoverable, for the sake of database consistency, and preferably cascadeless.
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency.
- Concurrency-control schemes tradeoff between the amount of concurrency they allow and the amount of overhead that they incur.
- Some schemes allow only conflict-serializable schedules to be generated, while others allow view-serializable schedules that are not conflict-serializable.

CONCURRENCY CONTROL VS. SERIALIZABILITY TEST

- Concurrency-control protocols allow concurrent schedules, but ensure that the schedules are conflict/view serializable, and are recoverable and cascadeless .
- Concurrency control protocols (generally) do not examine the precedence graph as it is being created
  - Instead a protocol imposes a discipline that avoids non-serializable schedules.
  - We study such protocols in Chapter 16.
- Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.
- Tests for serializability help us understand why a concurrency control protocol is correct.

WEAK LEVELS OF CONSISTENCY

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
  - E.g., a read-only transaction that wants to get an approximate total balance of all accounts
  - E.g., database statistics computed for query optimization can be approximate (why?)
  - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance

LEVELS OF CONSISTENCY IN SQL

- Serializable — default
- Repeatable read — only committed records to be read.
  - Repeated reads of same record must return same value.
  - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- Read committed — only committed records can be read.
  - Successive reads of record may return different (but committed) values.
- Read uncommitted — even uncommitted records may be read.
- Lower degrees of consistency useful for gathering approximate information about the database
- Warning: some database systems do not ensure serializable schedules by default

TRANSACTION DEFINITION IN SQL

- In SQL, a transaction begins implicitly.
- A transaction in SQL ends by:
  - Commit work commits current transaction and begins a new one.
  - Rollback work causes current transaction to abort.
- In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully
  - Implicit commit can be turned off by a database directive
- Isolation level can be set at database level
- Isolation level can be changed at start of transaction
  - E.g. In SQL **set transaction isolation level serializable**

IMPLEMENTATION OF ISOLATION LEVELS

- Locking
  - Lock on whole database vs lock on items
  - How long to hold lock?
  - Shared vs exclusive locks
- Timestamps
  - Transaction timestamp assigned e.g. when a transaction begins
  - Data items store two timestamps
    - Read timestamp
    - Write timestamp
  - Timestamps are used to detect out of order accesses
- Multiple versions of each data item
  - Allow transactions to read from a “snapshot” of the database

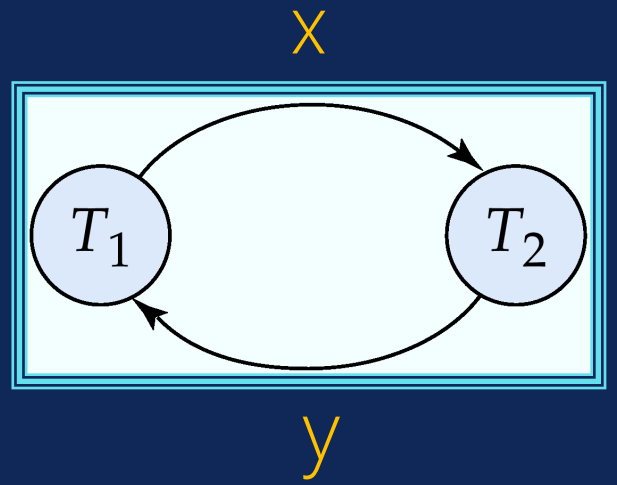
TRANSACTIONS AS SQL STATEMENTS

- E.g., Transaction 1: **select** *ID, name* **from** *instructor* **where** *salary* > 90000
- E.g., Transaction 2: **insert into** *instructor* **values** ('11111', 'James', 'Marketing', 100000)
- Suppose
  - T1 starts, finds tuples salary > 90000 using index and locks them
  - And then T2 executes.
  - Do T1 and T2 conflict? Does tuple level locking detect the conflict?
  - Instance of the phantom phenomenon
- Also consider T3 below, with Wu's salary = 90000

  update *instructor* **set** *salary* = *salary* * 1.1 **where** *name* = 'Wu'
- Key idea: Detect "predicate" conflicts, and use some form of "predicate locking"

TESTING FOR SERIALIZABILITY

- Consider some schedule of a set of transactions $T_1, T_2, \ldots, Tn$
- Precedence graph — a direct graph where the vertices are the transactions (names).
- We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- Example 1



TESTING FOR SERIALIZABILITY

- Example 1

T_1	T_2	T_3	T_4	T_5
	read(X)			
read(Y)				
read(Z)				
				read(V)
				read(W)
				read(W)
	read(Y)			
	write(Y)			
		write(Z)		
read(U)				
			read(Y)	
			write(Y)	
			read(Z)	
			write(Z)	
read(U)				
write(U)				

TESTING FOR CONFLICT SERIALIZABILITY

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order n^2 time, where *n* is the number of vertices in the graph. (Better algorithms take order $n + e$ where *e* is the number of edges.)
- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph. This is a linear order consistent with the partial order of the graph. For example, a serializability order for Schedule A would be

$$T_5 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$$

TESTING FOR VIEW SERIALIZABILITY

- The precedence graph test for conflict serializability must be modified to apply to a test for view serializability.
- The problem of checking if a schedule is view serializable falls in the class of *NP*-complete problems. Thus existence of an efficient algorithm is unlikely. However practical algorithms that just check some *sufficient conditions* for view serializability can still be used.

TESTING FOR VIEW SERIALIZABILITY

- Method-01:
- Check whether the given schedule is conflict serializable or not.
- If the given schedule is conflict serializable, then it is surely view serializable. Stop and report your answer.
- If the given schedule is not conflict serializable, then it may or may not be view serializable. Go and check using other methods.

TESTING FOR VIEW SERIALIZABILITY

- Method-02:
- Check if there exists any blind write operation.
  - (Writing without reading is called as a blind write).
- If there does not exist any blind write, then the schedule is surely not view serializable. Stop and report your answer.
- If there exists any blind write, then the schedule may or may not be view serializable. Go and check using other methods.

TESTING FOR VIEW SERIALIZABILITY

- Method-03:
- In this method, try finding a view equivalent serial schedule.
  - By using the above three conditions, write all the dependencies.
  - Then, draw a graph using those dependencies.
  - If there exists no cycle in the graph, then the schedule is view serializable otherwise not.

TESTING FOR RECOVERABLE SCHEDULE OR NOT

- Method-01:
- Check whether the given schedule is conflict serializable or not.
  - If the given schedule is conflict serializable, then it is surely recoverable. Stop and report your answer.
  - If the given schedule is not conflict serializable, then it may or may not be recoverable. Go and check using other methods.

TESTING FOR RECOVERABLE SCHEDULE OR NOT

- Method-02:
- Check if there exists any dirty read operation. (Reading from an uncommitted transaction is called as a dirty read)
  - If there does not exist any dirty read operation, then the schedule is surely recoverable. Stop and report your answer.
  - If there exists any dirty read operation, then the schedule may or may not be recoverable.
  - If there exists a dirty read operation, then follow the following cases-
    - Case-01: If the commit operation of the transaction performing the dirty read occurs before the commit or abort operation of the transaction which updated the value, then the schedule is irrecoverable.
    - Case-02: If the commit operation of the transaction performing the dirty read is delayed till the commit or abort operation of the transaction which updated the value, then the schedule is recoverable.

CONCURRENCY CONTROL VS SERIALIZABILITY TEST

- Testing a schedule for serializability *after* it has executed is a little too late!
- Goal – to develop concurrency control protocols that will assure serializability. They will generally not examine the precedence graph as it is being created; instead a protocol will impose a discipline that avoids nonseralizable schedules.
- Tests for serializability help understand why a concurrency control protocol is correct.

SERIAL VS SERIALIZABLE SCHEDULE

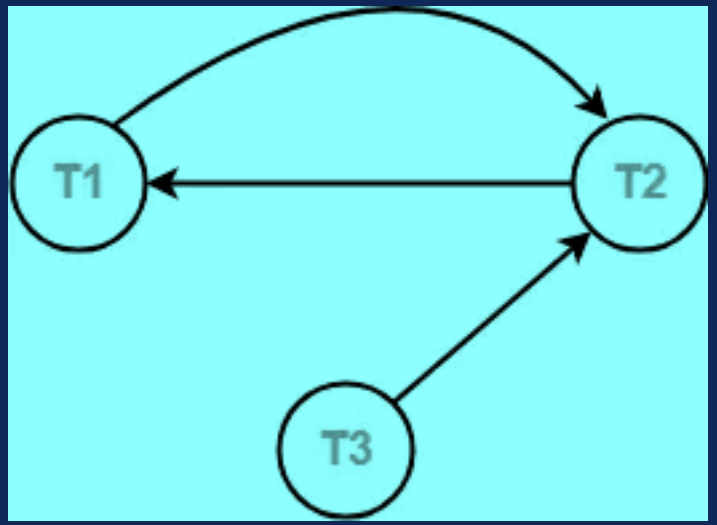
Serial Schedules	Serializable Schedules
No concurrency is allowed. Thus, all the transactions necessarily execute serially one after the other.	Concurrency is allowed. Thus, multiple transactions can execute concurrently.
Serial schedules lead to less resource utilization and CPU throughput.	Serializable schedules improve both resource utilization and CPU throughput.
Serial Schedules are less efficient as compared to serializable schedules. (due to above reason)	Serializable Schedules are always better than serial schedules. (due to above reason)

EXERCISE 1

- Check whether the given schedule S is conflict serializable or not-
  - S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$
- Solution:
- Step 1:
- List all the conflicting operations and determine the dependency between the transactions-
  - $R_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
  - $R_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
  - $R_3(B)$, $W_2(B)$ $(T_3 \rightarrow T_2)$

EXERCISE 1

- Check whether the given schedule S is conflict serializable or not-
  - S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$
- Solution:
- Step 2:
- Draw the precedence graph
  - $R_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
  - $R_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
  - $R_3(B)$, $W_2(B)$ $(T_3 \rightarrow T_2)$
- Clearly there exists a cycle in the graph.
- Therefore, the given schedule S is NOT CONFLICT SERIALIZABLE.



EXERCISE 2

- Check whether the given schedule S is conflict serializable and recoverable or not

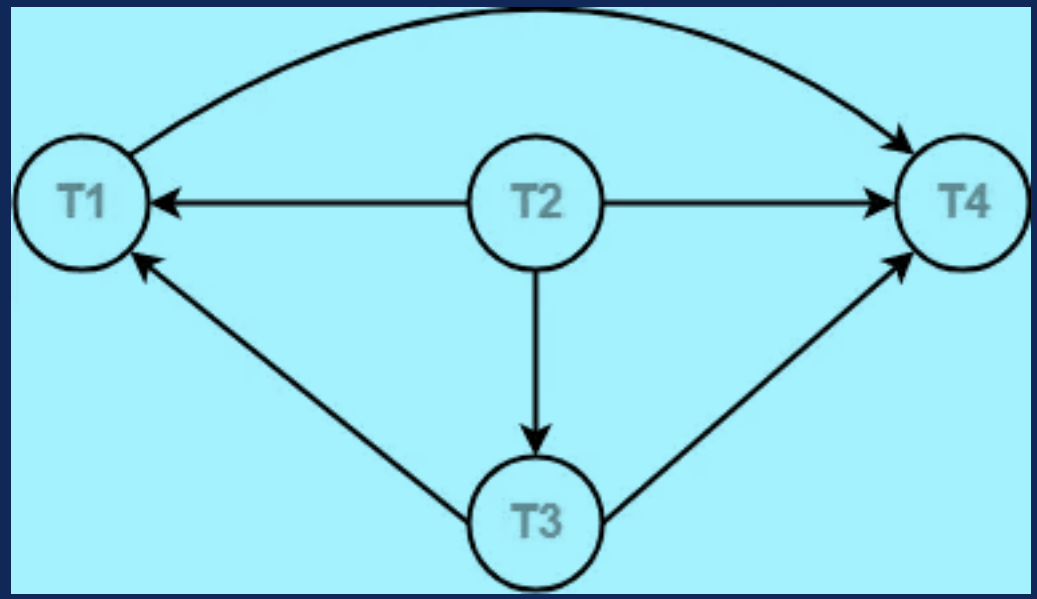
T1	T2	T3	T4
	R(X)		
		W(X)	
		Commit	
W(X)			
Commit			
	W(Y)		
	R(Z)		
	Commit		
			R(X)
			R(Y)
			Commit

EXERCISE 2

- Check whether the given schedule S is conflict serializable and recoverable or not
- Solution:
- Step 1:
- List all the conflicting operations and determine the dependency between the transactions-
  - $R_2(X)$, $W_3(X)$ $(T_2 \rightarrow T_3)$
  - $R_2(X)$, $W_1(X)$ $(T_2 \rightarrow T_1)$
  - $W_3(X)$, $W_1(X)$ $(T_3 \rightarrow T_1)$
  - $W_3(X)$, $R_4(X)$ $(T_3 \rightarrow T_4)$
  - $W_1(X)$, $R_4(X)$ $(T_1 \rightarrow T_4)$
  - $W_2(Y)$, $R_4(Y)$ $(T_2 \rightarrow T_4)$

EXERCISE 2

- Check whether the given schedule S is conflict serializable and recoverable or not
- Solution: Step 2:
- Draw the precedence graph
  - $R_2(X)$, $W_3(X)$ $(T_2 \rightarrow T_3)$
  - $R_2(X)$, $W_1(X)$ $(T_2 \rightarrow T_1)$
  - $W_3(X)$, $W_1(X)$ $(T_3 \rightarrow T_1)$
  - $W_3(X)$, $R_4(X)$ $(T_3 \rightarrow T_4)$
  - $W_1(X)$, $R_4(X)$ $(T_1 \rightarrow T_4)$
  - $W_2(Y)$, $R_4(Y)$ $(T_2 \rightarrow T_4)$



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S IS CONFLICT SERIALIZABLE.

EXERCISE 3

- Check whether the given schedule S is conflict serializable or not. If yes, then determine all the possible serialized schedules

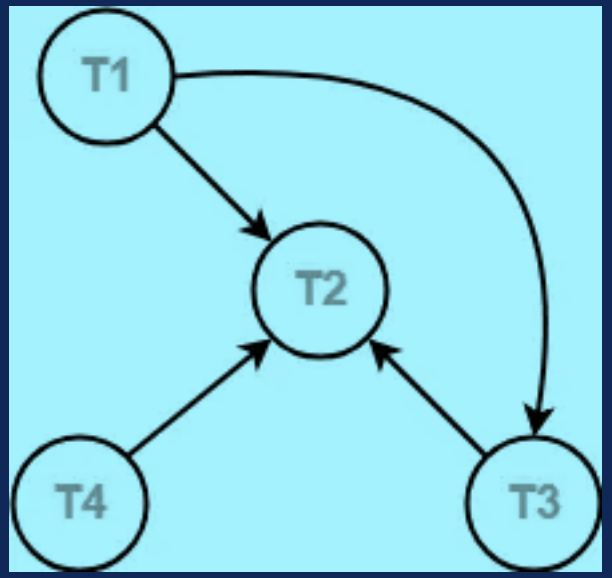
T1	T2	T3	T4
			R(A)
	R(A)		
		R(A)	
W(B)			
	W(A)		
		R(B)	
	W(B)		

EXERCISE 3

- Solution:
- Step 1: - Checking Whether S is conflict serializable or not?
- List all the conflicting operations and determine the dependency between the transactions-
  - $R_4(A)$, $W_2(A)$ $(T_4 \rightarrow T_2)$
  - $R_3(A)$, $W_2(A)$ $(T_3 \rightarrow T_2)$
  - $W_1(B)$, $R_3(B)$ $(T_1 \rightarrow T_3)$
  - $W_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
  - $R_3(B)$, $W_2(B)$ $(T_3 \rightarrow T_2)$

EXERCISE 3

- Solution:
- Step 1: - Checking Whether S is conflict serializable or not?
- Draw the precedence graph-
  - $R_4(A)$, $W_2(A)$ $(T_4 \rightarrow T_2)$
  - $R_3(A)$, $W_2(A)$ $(T_3 \rightarrow T_2)$
  - $W_1(B)$, $R_3(B)$ $(T_1 \rightarrow T_3)$
  - $W_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
  - $R_3(B)$, $W_2(B)$ $(T_3 \rightarrow T_2)$



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S IS CONFLICT SERIALIZABLE

EXERCISE 3

- Solution:
- Step 2: - Finding all the serializable schedules:
- All the possible topological orderings of the above precedence graph will be the possible serialized schedules.
- The topological orderings can be found by performing the Topological Sort of the above precedence graph.
- After performing the topological sort, the possible serialized schedules are-

1. $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$
2. $T_1 \rightarrow T_4 \rightarrow T_3 \rightarrow T_2$
3. $T_4 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2$

EXERCISE 4

- Check whether the given schedule S is view serializable or not

T1	T2	T3	T4
R (A)			
	R (A)		
		R (A)	
			R (A)
W (B)			
	W (B)		
		W (B)	
			W (B)

EXERCISE 4

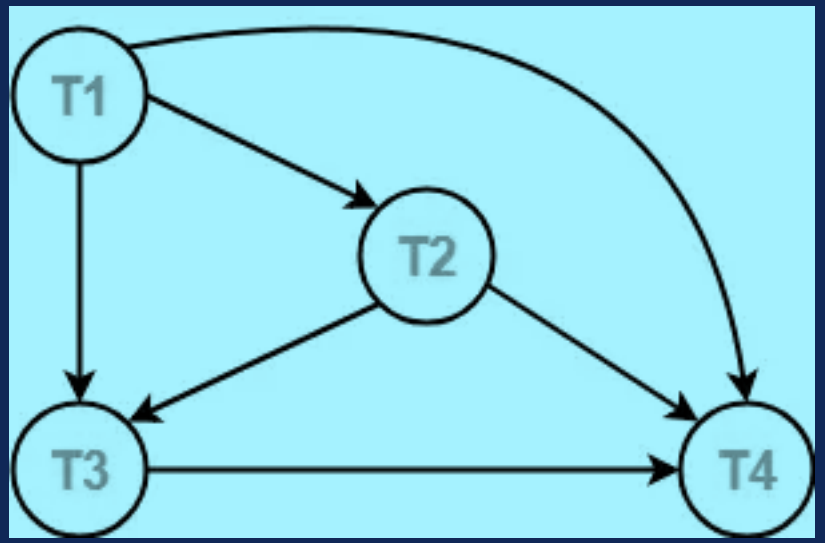
- Check whether the given schedule S is view serializable or not
- Solution-
- If a schedule is conflict serializable, then it is surely view serializable.
- So, let us check whether the given schedule is conflict serializable or not.
- Step-01:
- List all the conflicting operations and determine the dependency between the transactions-

EXERCISE 4

- Check whether the given schedule S is view serializable or not
- Solution-
- Step-01:
- List all the conflicting operations and determine the dependency between the transactions-
  - $W_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
  - $W_1(B)$, $W_3(B)$ $(T_1 \rightarrow T_3)$
  - $W_1(B)$, $W_4(B)$ $(T_1 \rightarrow T_4)$
  - $W_2(B)$, $W_3(B)$ $(T_2 \rightarrow T_3)$
  - $W_2(B)$, $W_4(B)$ $(T_2 \rightarrow T_4)$
  - $W_3(B)$, $W_4(B)$ $(T_3 \rightarrow T_4)$

EXERCISE 4

- Solution-
- Step-02:
- Draw the precedence graph-
  - $W_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
  - $W_1(B)$, $W_3(B)$ $(T_1 \rightarrow T_3)$
  - $W_1(B)$, $W_4(B)$ $(T_1 \rightarrow T_4)$
  - $W_2(B)$, $W_3(B)$ $(T_2 \rightarrow T_3)$
  - $W_2(B)$, $W_4(B)$ $(T_2 \rightarrow T_4)$
  - $W_3(B)$, $W_4(B)$ $(T_3 \rightarrow T_4)$



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S is conflict serializable.
- Thus, we conclude that the given schedule is also view serializable.

EXERCISE 5

- Check whether the given schedule S is view serializable or not

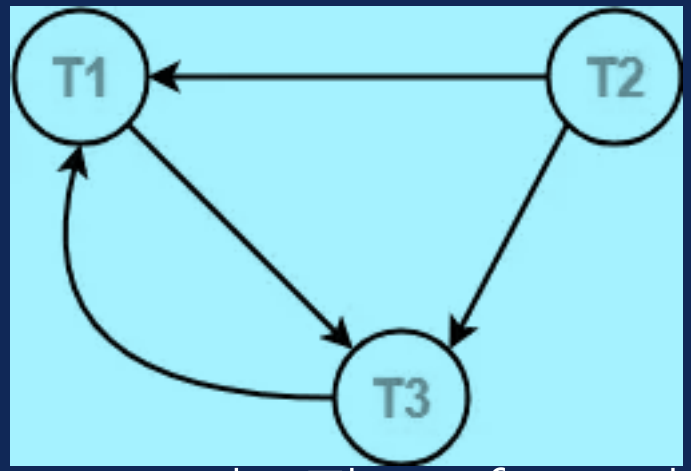
T1	T2	T3
R (A)		
	R (A)	
		W (A)
W (A)		

EXERCISE 5

- Solution-
- If a schedule is conflict serializable, then it is surely view serializable. So, let us check whether the given schedule is conflict serializable or not.
- Step-01:
- List all the conflicting operations and determine the dependency between the transactions-
  - $R_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$
  - $R_2(A)$, $W_3(A)$ $(T_2 \rightarrow T_3)$
  - $R_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
  - $W_3(A)$, $W_1(A)$ $(T_3 \rightarrow T_1)$

EXERCISE 5

- Solution- Step-02:
- Draw the precedence graph-
  - $R_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$
  - $R_2(A)$, $W_3(A)$ $(T_2 \rightarrow T_3)$
  - $R_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
  - $W_3(A)$, $W_1(A)$ $(T_3 \rightarrow T_1)$



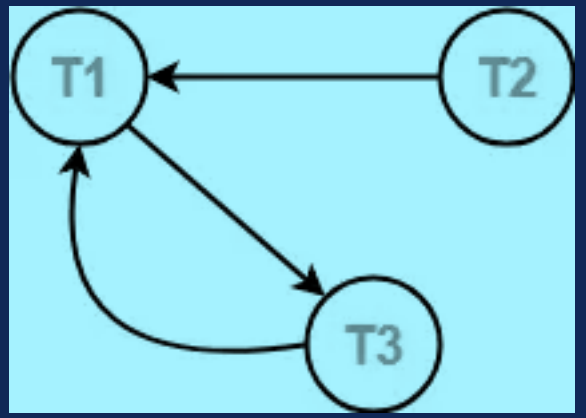
- learly, there exists a cycle in the precedence graph. Therefore, the given schedule S is not conflict serializable.
- Since, the given schedule S is not conflict serializable, so, it may or may not be view serializable.
- To check whether S is view serializable or not, let us use another method. **Let us check for blind writes.**

EXERCISE 5

- Solution-
- Step-03:
- Checking for Blind writes:
- There exists a blind write W3 (A) in the given schedule S.
- Therefore, the given schedule S may or may not be view serializable.
- To check whether S is view serializable or not, let us use another method.
- Let us derive the dependencies and then draw a dependency graph.

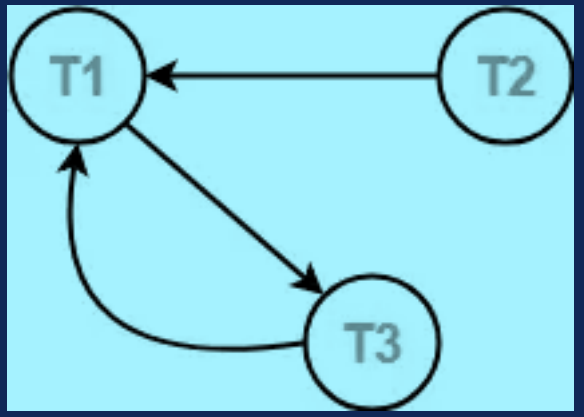
EXERCISE 5

- Solution-
- Step-04:
- Drawing a Dependency Graph:
- T1 firstly reads A and T3 firstly updates A.
- So, T1 must execute before T3.
- Thus, we get the dependency T1 → T3.
- Final updation on A is made by the transaction T1.
- So, T1 must execute after all other transactions.
- Thus, we get the dependency (T2, T3) → T1.
- There exists no write-read sequence.



EXERCISE 5

- Solution-
- Clearly, there exists a cycle in the dependency graph.
- Thus, we conclude that the given schedule S is not view serializable.



EXERCISE 6

- Check whether the given schedule S is view serializable or not. If yes, then give the serial schedule. S : $R_1(A)$, $W_2(A)$, $R_3(A)$, $W_1(A)$, $W_3(A)$

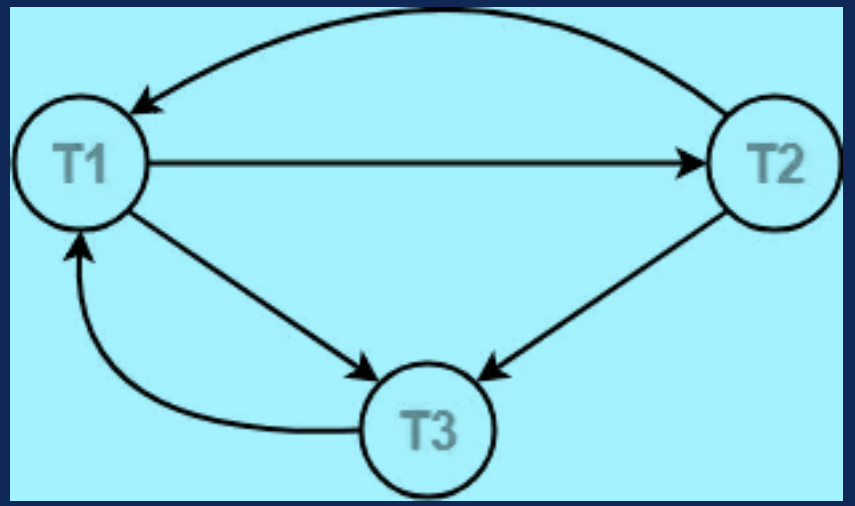
T1	T2	T3
R (A)		
	W (A)	
		R (A)
W (A)		
		W (A)

EXERCISE 6

- Solution-
- Step-01: Check whether the given schedule is conflict serializable or not.
- List all the conflicting operations and determine the dependency between the transactions-
  - $R_1(A)$, $W_2(A)$ $(T_1 \rightarrow T_2)$
  - $R_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$
  - $W_2(A)$, $R_3(A)$ $(T_2 \rightarrow T_3)$
  - $W_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
  - $W_2(A)$, $W_3(A)$ $(T_2 \rightarrow T_3)$
  - $R_3(A)$, $W_1(A)$ $(T_3 \rightarrow T_1)$
  - $W_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$

EXERCISE 6

- Solution- Step-02:
- Draw the precedence graph-
  - $R_1(A)$, $W_2(A)$ $(T_1 \rightarrow T_2)$
  - $R_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$
  - $W_2(A)$, $R_3(A)$ $(T_2 \rightarrow T_3)$
  - $W_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
  - $W_2(A)$, $W_3(A)$ $(T_2 \rightarrow T_3)$
  - $R_3(A)$, $W_1(A)$ $(T_3 \rightarrow T_1)$
  - $W_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$



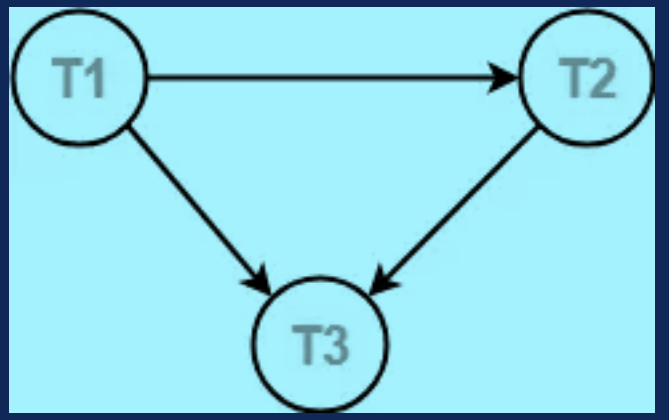
- Clearly, there exists a cycle in the precedence graph. Therefore, the given schedule S is not conflict serializable.
- Since, the given schedule S is not conflict serializable, so, it may or may not be view serializable.

EXERCISE 6

- Solution-
- Step-03: Checking for Blind writes:
- There exists a blind write W2 (A) in the given schedule S.
- Therefore, the given schedule S may or may not be view serializable.
- To check whether S is view serializable or not, let us use another method.
- Let us derive the dependencies and then draw a dependency graph.

EXERCISE 6

- Solution-
- Step-04: Drawing a Dependency Graph:
- T1 firstly reads A and T2 firstly updates A.
- So, T1 must execute before T2.
- Thus, we get the dependency T1 → T2.
- Final updation on A is made by the transaction T3.
- So, T3 must execute after all other transactions.
- Thus, we get the dependency (T1, T2) → T3.
- From write-read sequence, we get the dependency T2 → T3



EXERCISE 6

- Solution-

- Clearly, there exists no cycle in the dependency graph.
- Therefore, the given schedule S is view serializable.
- The serialization order T1 → T2 → T3.

EXERCISE 7

- Consider the following schedule is recoverable or not?

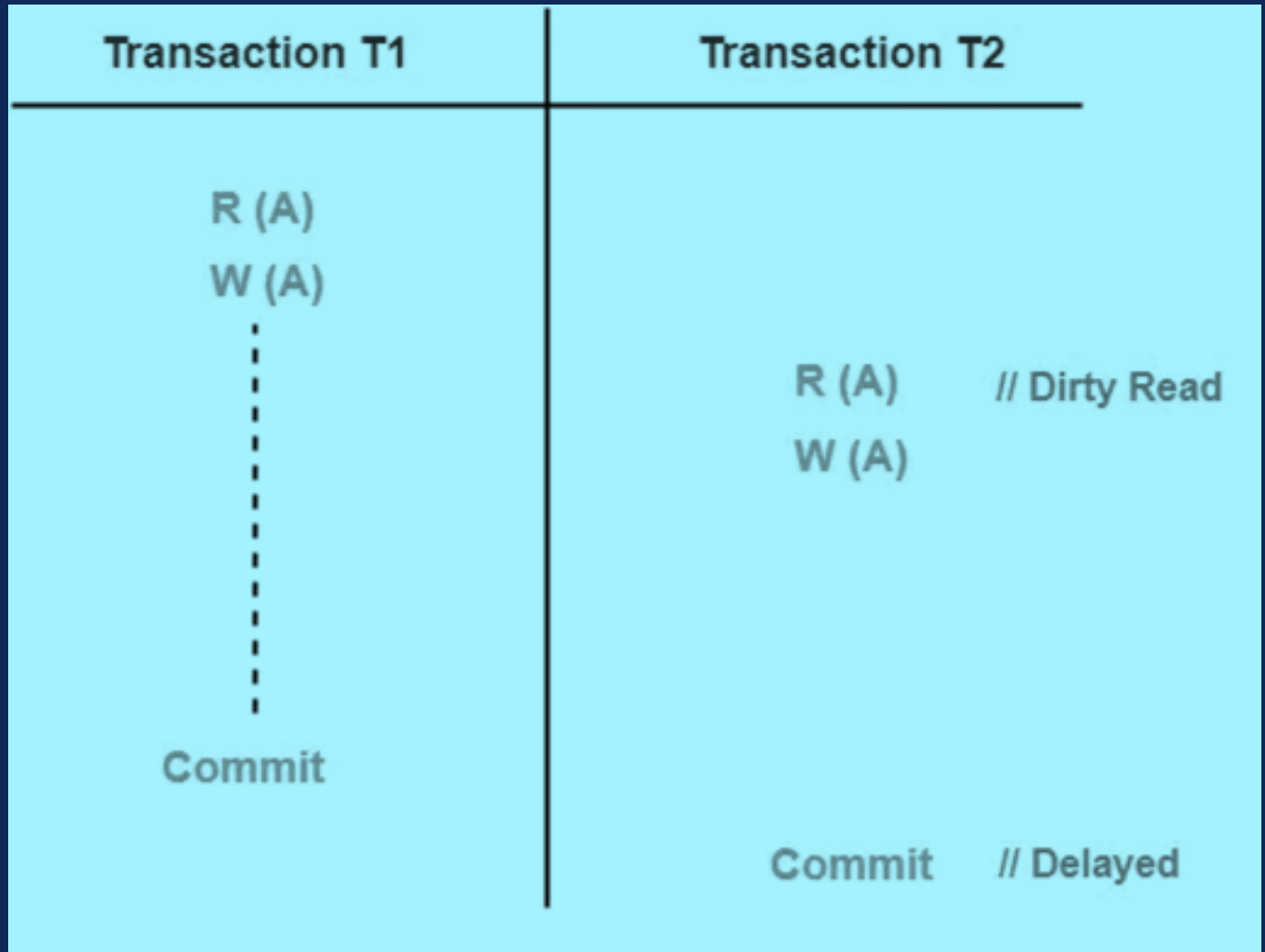
Transaction T1	Transaction T2
R (A)	
W (A)	
⋮	
	R (A) // Dirty Read
	W (A)
	Commit
Rollback	

EXERCISE 7

- Solution:
- T2 performs a dirty read operation.
- T2 commits before T1.
- T1 fails later and roll backs.
- The value that T2 read now stands to be incorrect.
- T2 can not recover since it has already committed.
- Hence Given Schedule S is irrecoverable.

EXERCISE 8

- Consider the following schedule is recoverable or not?



EXERCISE 8

- Solution:
- T2 performs a dirty read operation.
- The commit operation of T2 is delayed till T1 commits or roll backs.
- T1 commits later.
- T2 is now allowed to commit.
- In case, T1 would have failed, T2 has a chance to recover by rolling back.
- Hence Given Schedule S is recoverable.

EXERCISE 9

- Consider the following schedule are result equivalent?

Schedule S1

T1	T2
R (X)	
X = X + 5	
W (X)	
R (Y)	
Y = Y + 5	
W (Y)	
	R (X)
	X = X x 3
	W (X)

Schedule S2

T1	T2
R (X)	
X = X + 5	
W (X)	
	R (X)
	X = X x 3
	W (X)
R (Y)	
Y = Y + 5	
W (Y)	

Schedule S3

T1	T2
	R (X)
	X = X x 3
	W (X)
R (X)	
X = X + 5	
W (X)	
R (Y)	
Y = Y + 5	
W (Y)	

EXERCISE 9

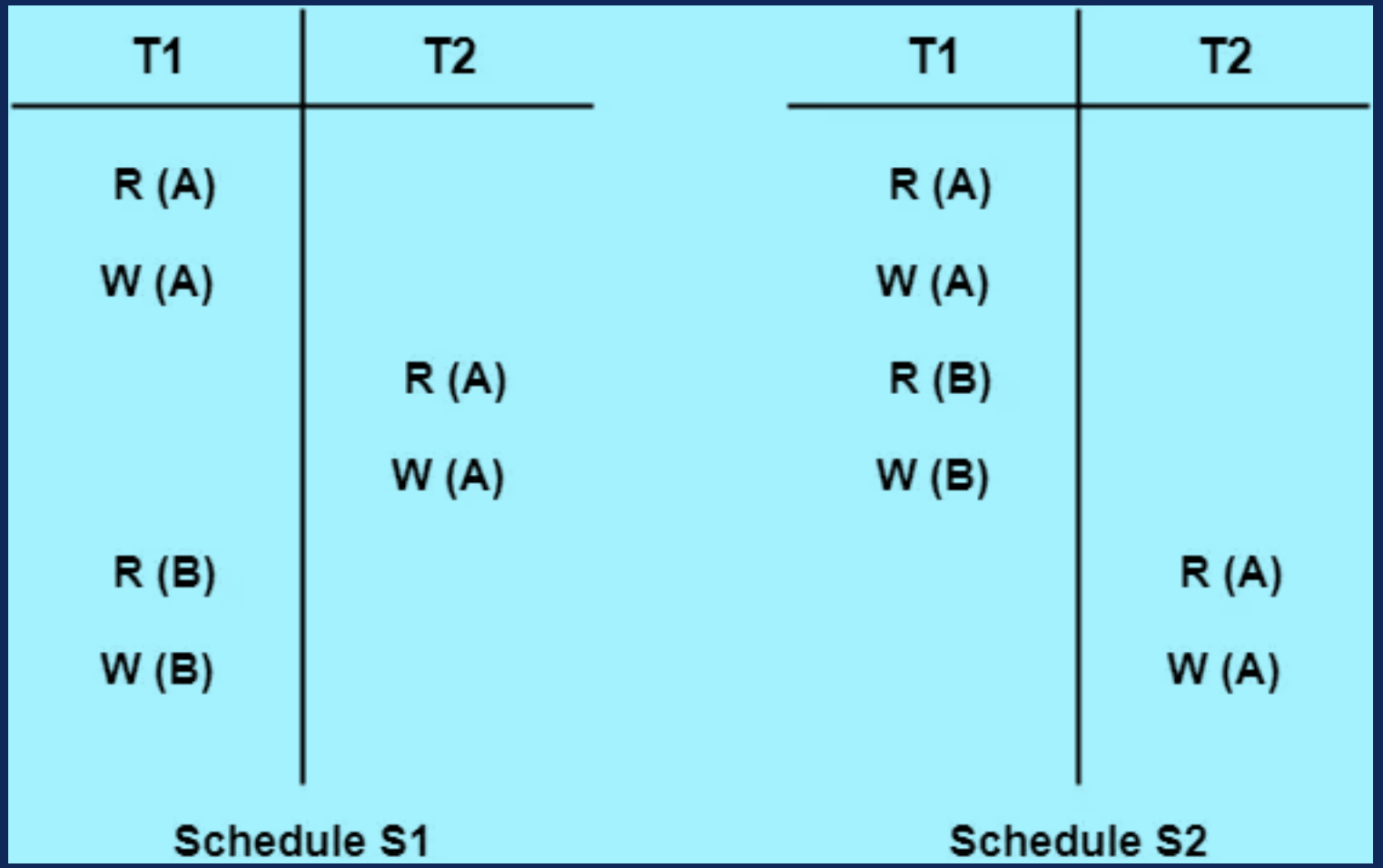
- Solution:
- To check whether the given schedules are result equivalent or not,
  - We will consider some arbitrary values of X and Y.
  - Then, we will compare the results produced by each schedule.
  - Those schedules which produce the same results will be result equivalent.
- Let X = 2 and Y = 5.

EXERCISE 9

- Solution:
- Let X = 2 and Y = 5.
- On substituting these values, the results produced by each schedule are
- Results by Schedule S1- X = 21 and Y = 10
- Results by Schedule S2- X = 21 and Y = 10
- Results by Schedule S3- X = 11 and Y = 10
- Clearly, the results produced by schedules S1 and S2 are same.
- Thus, we conclude that S1 and S2 are result equivalent schedules.

EXERCISE 10

- Consider the following schedule are conflict equivalent?



T1	T2
R (A)	
W (A)	
	R (A)
	W (A)
R (B)	
W (B)	

Schedule S1

T1	T2
R (A)	
W (A)	
R (B)	
W (B)	
	R (A)
	W (A)

Schedule S2

EXERCISE 10

- Solution:
- To check whether the given schedules are conflict equivalent or not,
  - We will write their order of pairs of conflicting operations.
  - Then, we will compare the order of both the schedules.
  - If both the schedules are found to have the same order, then they will be conflict equivalent.

EXERCISE 10

- Solution:
- For schedule S1- The required order is-
- $R_1(A)$, $W_2(A)$
- $W_1(A)$, $R_2(A)$
- $W_1(A)$, $W_2(A)$
- For schedule S2- The required order is-
- $R_1(A)$, $W_2(A)$
- $W_1(A)$, $R_2(A)$
- $W_1(A)$, $W_2(A)$
- Clearly, both the given schedules have the same order.
- Thus, we conclude that S1 and S2 are conflict equivalent schedules.

THANK YOU

BCSE304L – Database Systems

Module 5

Part 1